

CSP.01 Einsteins Rätsel als CSP

Variablen

Wir betrachten fünf Häuser in einer Reihe:

$$V = \{\text{Haus}_1, \text{Haus}_2, \text{Haus}_3, \text{Haus}_4, \text{Haus}_5\}.$$

Jedes Haus besitzt dieselben Attributtypen:

$$\text{Haus}_i = \{\text{Nummer}, \text{Farbe}, \text{Nationalität}, \text{Tier}, \text{Getränk}, \text{Zigaretten}\} \quad \text{mit } i \in \{1, \dots, 5\}.$$

Die Hausnummer ist fest:

$$\text{Haus}_i.\text{Nummer} = i.$$

Domänen

Für die Attributtypen definieren wir folgende Domänen:

$$D = \left\{ \begin{array}{l} D_{\text{Nummer}} = \{1, 2, 3, 4, 5\} \\ D_{\text{Farbe}} = \{\text{gelb, blau, rot, elfenbein, grün}\} \\ D_{\text{Nationalität}} = \{\text{Norweger, Ukrainer, Engländer, Spanier, Japaner}\} \\ D_{\text{Tier}} = \{\text{Fuchs, Pferd, Schnecken, Hund, Zebra}\} \\ D_{\text{Getränk}} = \{\text{Wasser, Tee, Milch, Orangensaft, Kaffee}\} \\ D_{\text{Zigaretten}} = \{\text{Kools, Chesterfield, OldGold, LuckyStrike, Parliament}\} \end{array} \right\}.$$

Jedes Attribut von Haus_i nimmt einen Wert aus der passenden Domäne an, z. B.

$$\text{Haus}_i.\text{Farbe} \in D_{\text{Farbe}}, \quad \text{Haus}_i.\text{Nationalität} \in D_{\text{Nationalität}}, \dots$$

Constraints

Wir schreiben die Constraints als Menge

$$C = \{c_1, c_2, \dots, c_{15}\}.$$

Wenn nicht anders angegeben, beziehen sich die genannten Attribute eines Constraints auf dasselbe Haus.

Struktur-Constraints (Eindeutigkeit)

$$\begin{aligned} c_1 : & \forall i \neq j : \text{Haus}_i.\text{Farbe} \neq \text{Haus}_j.\text{Farbe} \\ & \wedge \forall i \neq j : \text{Haus}_i.\text{Nationalität} \neq \text{Haus}_j.\text{Nationalität} \\ & \wedge \forall i \neq j : \text{Haus}_i.\text{Tier} \neq \text{Haus}_j.\text{Tier} \\ & \wedge \forall i \neq j : \text{Haus}_i.\text{Getränk} \neq \text{Haus}_j.\text{Getränk} \\ & \wedge \forall i \neq j : \text{Haus}_i.\text{Zigaretten} \neq \text{Haus}_j.\text{Zigaretten}. \end{aligned}$$

Unäre Constraints

$$c_2 : \text{Haus}_3.\text{Getränk} = \text{Milch}$$

$$c_3 : \text{Haus}_1.\text{Nationalität} = \text{Norweger}.$$

Binäre Constraints im selben Haus

$$c_4 : (\text{Nationalität}, \text{Farbe}), \quad \text{Engländer} = \text{rot}, \\ \text{d. h. } \forall i : (\text{Haus}_i.\text{Nationalität} = \text{Engländer} \Rightarrow \text{Haus}_i.\text{Farbe} = \text{rot})$$

$$c_5 : (\text{Nationalität}, \text{Tier}), \quad \text{Spanier} = \text{Hund}$$

$$c_6 : (\text{Farbe}, \text{Getränk}), \quad \text{grün} = \text{Kaffee}$$

$$c_7 : (\text{Nationalität}, \text{Getränk}), \quad \text{Ukrainer} = \text{Tee}$$

$$c_8 : (\text{Zigaretten}, \text{Tier}), \quad \text{OldGold} = \text{Schnecken}$$

$$c_9 : (\text{Zigaretten}, \text{Farbe}), \quad \text{Kools} = \text{gelb}$$

$$c_{10} : (\text{Zigaretten}, \text{Getränk}), \quad \text{LuckyStrike} = \text{Orangensaft}$$

$$c_{11} : (\text{Nationalität}, \text{Zigaretten}), \quad \text{Japaner} = \text{Parliament}.$$

Binäre Constraints zwischen verschiedenen Häusern

Hier benutzen wir explizit zwei Haus-Variablen h_1, h_2 .

$$c_{12} : (h_g = \text{Haus}(\text{Farbe} = \text{grün}), h_e = \text{Haus}(\text{Farbe} = \text{elfenbein})), \\ h_g.\text{Nummer} = h_e.\text{Nummer} + 1 \quad (\text{grün steht direkt rechts von elfenbein})$$

$$c_{13} : (h_c = \text{Haus}(\text{Zigaretten} = \text{Chesterfield}), h_f = \text{Haus}(\text{Tier} = \text{Fuchs})), \\ |h_c.\text{Nummer} - h_f.\text{Nummer}| = 1 \quad (\text{Chesterfield wohnt neben Fuchs})$$

$$c_{14} : (h_k = \text{Haus}(\text{Zigaretten} = \text{Kools}), h_p = \text{Haus}(\text{Tier} = \text{Pferd})), \\ |h_k.\text{Nummer} - h_p.\text{Nummer}| = 1 \quad (\text{Kools neben Pferd})$$

$$c_{15} : (h_n = \text{Haus}(\text{Nationalität} = \text{Norweger}), h_b = \text{Haus}(\text{Farbe} = \text{blau})), \\ |h_n.\text{Nummer} - h_b.\text{Nummer}| = 1 \quad (\text{Norweger neben blauem Haus}).$$

Damit ist das Einstein-Rätsel als CSP

$$\langle V, D, C \rangle$$

mit Variablen, Domänen sowie unären und binären Constraints formalisiert.

CSP.02: Framework für Constraint Satisfaction

Datei: `zebra.csp.py`

CSP.03: Kantenkonsistenz mit AC-3

Gegeben sei das Constraint Satisfaction Problem

$$\langle \{v_1, v_2, v_3, v_4\}, \{D_{v_1} = D_{v_2} = D_{v_3} = D_{v_4} = D\}, \{c_1, c_2, c_3, c_4\} \rangle$$

mit der Domäne

$$D = \{0, 1, 2, 3, 4, 5\}$$

und den binären Constraints

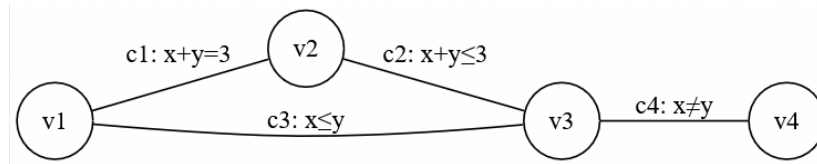
$$c_1 = ((v_1, v_2), \{(x, y) \mid x + y = 3\}),$$

$$c_2 = ((v_2, v_3), \{(x, y) \mid x + y \leq 3\}),$$

$$c_3 = ((v_1, v_3), \{(x, y) \mid x \leq y\}),$$

$$c_4 = ((v_3, v_4), \{(x, y) \mid x \neq y\}).$$

Constraint-Graph:



Die Kanten stehen jeweils für die oben definierten Constraints c_1 bis c_4 .

AC-3: Schrittweise Ausführung

Zu Beginn sind alle Domänen gleich:

$$D(v_1) = D(v_2) = D(v_3) = D(v_4) = \{0, 1, 2, 3, 4, 5\}.$$

Die initiale Queue aller gerichteten Bögen lautet:

$$Q_0 = \{(v_1, v_2), (v_2, v_1), (v_2, v_3), (v_3, v_2), (v_1, v_3), (v_3, v_1), (v_3, v_4), (v_4, v_3)\}.$$

Im Folgenden werden für jede relevante Iteration der aktuelle Bogen, der Queue-Zustand und das Ergebnis von `ARC_REDUCE` angegeben.

Iteration 1: Bogen (v_1, v_2) , **Constraint** $c_1 : x + y = 3$

Queue vor Schritt:

$$[(v_1, v_2), (v_2, v_1), (v_2, v_3), (v_3, v_2), (v_1, v_3), (v_3, v_1), (v_3, v_4), (v_4, v_3)]$$

Prüfung und Reduktion: Für jedes $x \in D(v_1)$ muss ein $y \in D(v_2)$ existieren mit $x + y = 3$.

Ungültig: $x = 4$ und $x = 5$. Daher:

$$D(v_1) = \{0, 1, 2, 3\}.$$

Neue Kanten in Queue: (v_3, v_1) .

Iteration 2: Bogen (v_2, v_1) , **wieder** c_1

Für jedes $y \in D(v_2)$ muss ein $x \in D(v_1)$ existieren mit $x + y = 3$.

Ungültig: $y = 4$, $y = 5$. Damit:

$$D(v_2) = \{0, 1, 2, 3\}.$$

Neue Kante: (v_3, v_2) .

Iteration 3: Bogen (v_2, v_3) , **Constraint** $c_2 : x + y \leq 3$

Alle Werte in $D(v_2) = \{0, 1, 2, 3\}$ haben passende Werte in $D(v_3)$.

$D(v_2)$ bleibt unverändert.

Iteration 4: Bogen (v_3, v_2) , **wieder** c_2

Für jedes $x \in D(v_3)$ muss es $y \in D(v_2)$ geben mit $x + y \leq 3$.

Ungültige Werte: 4 und 5.

$$D(v_3) = \{0, 1, 2, 3\}.$$

Neue Kanten: (v_1, v_3) und (v_4, v_3) .

Iteration 5: Bogen (v_1, v_3) , **Constraint** $c_3 : x \leq y$

Für jedes $x \in D(v_1) = \{0, 1, 2, 3\}$ existiert ein $y \in D(v_3) = \{0, 1, 2, 3\}$ mit $x \leq y$.

$D(v_1)$ bleibt unverändert.

Iteration 6: Bogen (v_3, v_1) , wieder c_3

Zu jedem $x \in D(v_3)$ existiert ein $y \in D(v_1)$ mit $y \leq x$.

$D(v_3)$ bleibt unverändert.

Iteration 7 und 8: Bögen (v_3, v_4) und (v_4, v_3) , Constraint $c_4 : x \neq y$

Alle Werte in beiden Domänen haben mindestens einen ungleichen Partner.

$D(v_4)$ bleibt unverändert.

Endergebnis der AC-3 Konsistenzprüfung

Nach Abschluss aller Schritte ergibt sich der kantenkonsistente Zustand:

$$\begin{aligned} D(v_1) &= \{0, 1, 2, 3\}, \\ D(v_2) &= \{0, 1, 2, 3\}, \\ D(v_3) &= \{0, 1, 2, 3\}, \\ D(v_4) &= \{0, 1, 2, 3, 4, 5\}. \end{aligned}$$

Eine vollständige Lösung liefert AC-3 hier nicht; jedoch erfolgt eine deutliche Domänenreduktion.

CSP.04: Forward Checking und Kantenkonsistenz

Wir betrachten wieder das CSP aus Aufgabe CSP.03. Die partielle Belegung lautet

$$\alpha = \{v_1 \mapsto 2\}.$$

Damit setzen wir

$$D(v_1) = \{2\}, \quad D(v_2) = D(v_3) = D(v_4) = \{0, 1, 2, 3, 4, 5\}.$$

(1) Kantenkonsistenz in α

Durch das Prüfen aller beteiligten Constraints werden Werte entfernt, die keinen passenden Partner mehr besitzen.

Variable	Domäne vorher	Domäne nach Kantenkonsistenz
v_1	$\{2\}$	$\{2\}$
v_2	$\{0, 1, 2, 3, 4, 5\}$	$\{1\}$
v_3	$\{0, 1, 2, 3, 4, 5\}$	$\{2\}$
v_4	$\{0, 1, 2, 3, 4, 5\}$	$\{0, 1, 3, 4, 5\}$

Durch Kantenkonsistenz werden also die Domänen von v_2 , v_3 und v_4 deutlich kleiner.

(2) Forward Checking in α

Forward Checking prüft nur die Constraints, die direkt die bereits belegte Variable v_1 betreffen.

Variable	Domäne vorher	Domäne nach Forward Checking
v_1	$\{2\}$	$\{2\}$
v_2	$\{0,1,2,3,4,5\}$	$\{1\}$
v_3	$\{0,1,2,3,4,5\}$	$\{2,3,4,5\}$
v_4	$\{0,1,2,3,4,5\}$	$\{0,1,2,3,4,5\}$

Vergleich

Kantenkonsistenz entfernt mehr Werte, da sie alle betroffenen Bögen berücksichtigt. Forward Checking schaut nur einen Schritt voraus und schränkt deshalb weniger ein. Kantenkonsistenz ist dadurch stärker, aber auch etwas aufwendiger.

CSP.05: Planung von Indoor-Spielplätzen als CSP

Wir abstrahieren den Spielplatz als rechteckiges Gitter. Zur Vereinfachung nehmen wir z. B. ein

$$8 \times 5\text{-Raster} \quad (x \in \{1, \dots, 8\}, y \in \{1, \dots, 5\}).$$

Jedes Spielgerät belegt eine zusammenhängende rechteckige Teilfläche des Gitters. Türen liegen am Rand und werden durch einige feste Gitterzellen beschrieben, die frei bleiben müssen.

Modellierung als CSP

Wir betrachten vier Bereiche:

- Bar B (rechteckig, z. B. 2×2 Felder),
- Hüpfburg H (z. B. 2×3 Felder),
- Kletterberg K (z. B. 2×2 Felder),
- Go-Kart-Bahn G (z. B. 3×2 Felder).

Jedes Gerät hat feste Abmessungen. Die Position wird durch die linke obere Ecke seiner Rechtecksfläche angegeben.

Variablen

$$V = \{P_B, P_H, P_K, P_G\},$$

wobei jede Variable P_X die Gitterposition der linken oberen Ecke des Geräts X beschreibt.

Domänen Die Domäne D_{P_X} enthält alle Gitterpaare (x, y) , für die das Rechteck von X

- vollständig im 8×5 -Raster liegt und
- keine Türzelle überdeckt.

Beispiel (informell):

$$D_{P_B} \subseteq \{(x, y) \mid 1 \leq x \leq 7, 1 \leq y \leq 4\},$$

analog für P_H, P_K, P_G (je nach Breite/Höhe).

Für die Bar bevorzugen wir die Nähe zum Eingang, z. B. Tür am linken Rand:

$$D_{P_B} = \{(x, y) \in D_{P_B}^{\text{roh}} \mid x \leq 2\},$$

also ein unärer Constraint „Bar nahe Eingang“.

Constraints Sei $\text{Rect}(P_X)$ die Menge aller Gitterzellen, die von Gerät X belegt werden, und $\text{Rect}^{+1}(P_X)$ die belegte Fläche plus einem Sicherheitsrand von einer Zelle (1 m).

- **Nicht-Überlappung und Sicherheitsabstand** Für alle Paare $X \neq Y$:

$$C_{\text{dist}}(P_X, P_Y) : \quad \text{Rect}^{+1}(P_X) \cap \text{Rect}^{+1}(P_Y) = \emptyset.$$

Die Geräte überlappen sich nicht und haben mindestens 1 m Abstand.

- **Notausgänge freihalten** Alle Zellen, die zu Notausgängen gehören, sind aus den Domänen entfernt (unäre Constraints auf P_X).
- **Sichtlinie Bar $\rightarrow$ Kletterberg** Zur Vereinfachung: Bar und Kletterberg liegen in derselben Zeile, und kein anderes Gerät steht dazwischen:

$$C_{\text{sicht}}(P_B, P_K, P_H, P_G) :$$

Bar und Kletterberg teilen sich eine Zeile, und für diese Zeile schneidet kein anderes Rechteck $\text{Rect}(P_H)$ oder $\text{Rect}(P_G)$ den Bereich zwischen Bar und Kletterberg.

- **Entspannungszonen** Man kann z. B. fordern, dass um die Bar herum mindestens ein freies Feld bleibt (wieder als Abstand in $\text{Rect}^{+1}(P_X)$ modellierbar).

Damit erhält man formal ein CSP

$$\langle V, D, C \rangle$$

mit $V = \{P_B, P_H, P_K, P_G\}$, Domänen D_{P_X} wie oben beschrieben und den Constraints $C_{\text{dist}}, C_{\text{sicht}}$ sowie den unären Constraints für Türen und Bar-Nähe.

Lösen mit MAC

Für eine Backtracking-Lösung können wir pro Schritt eine Variable P_X auswählen (z. B. mit MRV) und ihr eine Position aus D_{P_X} zuweisen. Mit MAC (Maintaining Arc Consistency) wird nach jeder Zuweisung Kantenkonsistenz zwischen P_X und allen noch unbelegten Variablen hergestellt. Dadurch werden viele ungültige Positionen früh entfernt, z. B. alle Plätze, die den Sicherheitsabstand verletzen oder Türen blockieren. So findet der Backtracking-Algorithmus schneller eine vollständige, konsistente Belegung aller P_X .

Lösen mit Min-Conflicts

Alternativ kann man eine lokale Suche mit Min-Conflicts verwenden. Ein Zustand ist hier eine vollständige Zuweisung

$$\{P_B = (x_B, y_B), P_H = (x_H, y_H), P_K = (x_K, y_K), P_G = (x_G, y_G)\}$$

mit gültigen Gitterpositionen.

- Start: zufällige gültige Positionen aus den Domänen wählen.
- Konflikte: zählen, wie viele Constraints verletzt sind, z. B. Überlappungen, Abstandverletzungen oder schlechte Sichtlinie.
- Schritt: eine Variable P_X wählen, die an Konflikten beteiligt ist, und innerhalb von D_{P_X} die Position wählen, die die Anzahl der Konflikte minimal macht.

Durch wiederholtes Anwenden dieses Schritts kann Min-Conflicts in vielen Fällen schnell eine Konfiguration finden, in der alle Geräte einen gültigen Abstand haben, Türen frei sind und die Sicht von der Bar auf den Kletterberg möglich ist.